AI-Assisted Release Notes Generator-Prompt Workflow Design

Student: T. Pranadeep Reddy

Registration No.: 99230041004

Department: CSE-AIML

Assessment: Mini Assessment - Prompt Workflow Design

1. Introduction

This report documents a multi-stage prompting pipeline built to convert a raw, developer-written git commit log into a clean, customer-facing release notes page. Commit messages are written in a hurry, use internal shorthand, and mix trivial changes with breaking ones, which makes them unusable as-is for end users. Rather than firing a single "make release notes from this" instruction at the model, the task is divided into three linked stages. Each stage consumes and validates the previous stage's output, so the published notes end up both technically accurate and consistently formatted.

2. Objective

The aim is to design and execute a multi-step prompt sequence that classifies scattered commit messages into standard release-note categories, drafts a formatted release page under explicit style and length rules, and then subjects that draft to a self-review against a fixed checklist so a corrected, publishable final version can be produced.

3. Scenario Selected

Technical Documentation track: transform unpolished engineering artifacts (commit logs) into a structured, user-readable document, then run a self-audit step to catch omitted, mislabelled, or under-explained changes before release.

4. Prompt Workflow

Step	Process	Output
1	Raw Commit Log Collection	Unstructured git log / PR titles
2	Prompt 1 - Extraction & Categorisation	Structured 4-category outline
3	Prompt 2 - Drafting	First draft of release notes
4	Prompt 3 - Critique & Refinement	Self-critique + final release notes

Overall flow: Raw Commit Log → Prompt 1 (Extraction & Categorisation) → Prompt 2 (Drafting) → Prompt 3 (Critique & Refinement) → Final Release Notes.

5. Example Input (Raw Commit Log)

.v2.4.0 commit dump -- messy, unfiltered

- feat(auth): swap session cookies for JWT, old /login response shape gone (#412)
- fix: null ptr crash when profile pic missing, users saw white screen (#418)
- feat: bulk CSV export on reports page, was top user request (#421)
- chore: bump lodash, eslint, jest -- no user impact
- perf: cache dashboard queries, load time 4.2s to 0.9s (#427)
- fix(api): pagination returned dup rows on page 2+ (#430)
- BREAKING: /v1/users endpoint removed, use /v2/users, deprecation warned since 2.1
- feat: dark mode toggle in settings (#433)

- known issue: CSV export times out over 50k rows, fix planned 2.4.1
- docs: fixed typos in readme
- refactor internal logger, nothing user facing

6. Prompt Chain Transcript & Intermediate AI Responses

Step 1 - Extraction & Categorisation

prompt 1: Act as a technical writer preparing release notes for a SaaS product. I will paste a raw git commit log for version 2.4.0. Filter out anything with no user impact (chores, refactors, internal docs) and sort the remaining entries into exactly four categories: New Features, Improvements, Bug Fixes, and Breaking Changes & Known Issues. Return only a structured outline. Keep the tone neutral and professional.

AI Response 1 (Outline):

- New Features - bulk CSV export on the reports page (#421); dark mode toggle in settings (#433).
- Improvements - dashboard query caching, load time reduced from 4.2s to 0.9s (#427); authentication migrated from session cookies to JWT (#412).
- Bug Fixes - blank screen crash when a profile picture is absent (#418); duplicate rows returned by paginated API responses (#430).
- Breaking Changes & Known Issues - /v1/users endpoint removed in favour of /v2/users (deprecated since 2.1); /login response shape changed by the JWT migration; CSV export times out beyond 50,000 rows, fix scheduled for 2.4.1.
- Excluded (no user impact) - dependency bumps, logger refactor, README typo fixes.

Step 2 - Generation & Drafting

Prompt 2:

Good. Using nothing but the outline produced above, write the complete v2.4.0 release notes page. Rules to follow: use the four category headings in the outline's order; maximum three bullets per category; one plain-English sentence per bullet describing the user-visible effect; append the issue number to every entry; give a migration hint for each breaking change; stay under 300 words total.

AI Response 2 (Draft Release Notes):

Release Notes - Version 2.4.0

New Features

- You can now export any report to CSV in bulk directly from the Reports page. (#421)

- A dark mode toggle has been added to Settings. (#433)

Improvements

- Dashboard pages now load in under a second, down from roughly four seconds, thanks to query caching. (#427)

- Authentication now uses JWT tokens instead of session cookies, improving stability across devices. (#412)

Bug Fixes

- Fixed a crash that showed a blank white screen when a user had no profile picture. (#418)

- Fixed duplicate records appearing on the second and later pages of API results. (#430)

Breaking Changes

- The /v1/users endpoint has been removed. Migrate your integrations to /v2/users; the response schema is unchanged apart from the version prefix.

- The /login response shape has changed. (#412)

Step 3 - Evaluation & Refinement (Critique Gate)

Prompt 3: Before we publish, act as a strict release manager reviewing this draft. Check it against the following checklist:

- Are all four categories from the outline present?

- Does every category stay within the three-bullet limit?

- Does every entry carry its issue number?

- Does every breaking change include a concrete migration hint?

- Is the total word count under 300 words?

First, write out your critique and flag anywhere the draft falls short. Second, produce the corrected, final version of the release notes.

AI Response 3 (Critique + Final Version):

Critique: Three of the four categories are correct, within the bullet cap, and carry issue numbers. Two failures were found. First, the heading was shortened to "Breaking Changes" and the Known Issue from the outline, the CSV export timeout above 50,000 rows, was dropped entirely; since it directly affects the headline new feature, omitting it is misleading. Second, the /login breaking change states what changed but gives no migration hint, violating the checklist. Word count is within the limit, leaving room for both corrections.

Final Release Notes:

Release Notes - Version 2.4.0

New Features

- Bulk CSV export is now available directly from the Reports page. (#421)

- A dark mode toggle has been added to Settings. (#433)

Improvements

- Dashboards now load in about 0.9 seconds instead of 4.2 seconds, thanks to query caching. (#427)

- Authentication now uses JWT tokens instead of session cookies for better cross-device stability. (#412)

Bug Fixes

- Fixed a blank-screen crash affecting users without a profile picture. (#418)

- Fixed duplicate records returned on page 2 and beyond of paginated API responses. (#430)

Breaking Changes & Known Issues

- /v1/users has been removed after deprecation in 2.1. Repoint integrations to /v2/users; the payload is otherwise identical.

- The /login response now returns a token and expiry in the body instead of a session cookie. Read the token from the body and send it as an Authorization Bearer header. (#412)

- Known issue: CSV exports above 50,000 rows may time out; split the export by date range until the fix ships in 2.4.1.

Chat Link/Screenshot: [Paste the link to your live chat session or attach a screenshot of the full conversation thread here before submission.]

7. Chaining & Dependency Analysis

Each stage is explicitly bound to the one before it. Prompt 1 performs classification and filtering only, and never writes prose. Prompt 2 is instructed to work solely from the outline generated in Prompt 1, so any commit filtered out at Step 1 cannot silently reappear, and any category defined at Step 1 must be honoured. Prompt 3 then evaluates that exact draft against a fixed, objective checklist before emitting a corrected version. This forms a genuine critique gate: the model is forced to compare two of its own artifacts (outline versus draft) and detect drift, rather than simply restating the draft in different words. The dropped Known Issues entry is proof that the gate does real work, since a single-shot prompt would have shipped that omission straight to users.

8. Advantages of This Workflow

- Splits a large task into inspectable stages, so an error can be traced to the exact step that introduced it instead of hiding inside one opaque single-shot prompt.
- The filtering decision about what is not user-facing is made once, early, and is then enforced downstream by every later stage.
- The critique gate catches silent omissions and rule violations, here a missing known issue and a missing migration hint, before publication.
- Converts an engineering-only artifact into a customer-ready document with consistent tone, structure, and traceable issue references.
- The pattern is reusable for every subsequent release; only the raw commit log changes, while all three prompts stay fixed.

9. Conclusion

This three-stage prompt chain demonstrates how a noisy, developer-oriented commit log can be converted into dependable, user-facing release notes through

explicit dependency between stages and a mandatory self-critique step. The workflow satisfies the assignment's chaining, dependency, and critique-gate requirements, while remaining generic enough to be applied to any release cycle, changelog, or technical document produced from raw engineering output.